

Динамическое изменение параметров модуля ядра

Владислав Богданов

2026-07-09

Оглавление

1. Введение	1
2. param_set/param_get	1
2.1. Описание	1
2.2. Как написать свои param_set и param_get	2
2.2.1. Параметр с валидацией диапазона	2
2.2.2. Параметр с callback'ом (реакция на изменение)	4
2.2.3. Когда использовать param_set/param_get	7
2.2.4. Ключевые моменты	7
2.2.5. Продвинутый трюк: массив с callback'ом	7
3. Sysfs	8
3.1. Описание	9
3.2. Использование	10
4. /proc файлы	10
4.1. Описание	10
4.2. Использование	11
4.3. /proc файлы для нескольких параметров	12
4.3.1. Поддиректория с отдельными файлами (рекомендуемый)	12
4.3.2. Использование	16
4.4. Один файл с командами (упрощенный)	17
4.4.1. Использование	20
5. debugfs (для отладки)	20
5.1. Описание	20
5.2. Использование	21
6. Character Device + ioctl	21
6.1. Описание	21
6.2. Пользовательская программа	23
6.3. Использование	24
7. Netlink сокет	24
7.1. Описание	24
7.2. Пользовательская программа	25
7.3. Использование	26
8. configfs (для сложной конфигурации)	26
8.1. Описание	26
8.2. Использование	28

1. Введение

При загрузке модуля ядро автоматически создает файлы в `/sys/module/<module_name>/parameters`. Эти файлы просто хранят значения переменных. Когда меняем значение через `echo`, переменная в памяти обновляется, но никакой код не выполняется. Для того что бы поменять параметры и перезапустить логику модуля ядра есть несколько способов.

Способы:

Метод	Сложность	Гибкость	Когда использовать
<code>param_set/param_get</code>	Низкая	Низкая	Простые данные
<code>sysfs</code>	Низкая	Средняя	Стандартный выбор для параметров
<code>/proc</code>	Низкая	Высокая	Устарел, но еще встречается
<code>debugfs</code>	Очень низкая	Низкая	Только для отладки
<code>ioctl</code>	Высокая	Очень высокая	Сложные структуры, бинарные данные
<code>netlink</code>	Очень высокая	Очень высокая	Сетевые подсистемы, уведомления
<code>configfs</code>	Очень высокая	Очень высокая	Конфигурация сложных объектов

Возможно комбинирование спосоов.

2. `param_set/param_get`

2.1. Описание

Это функции ядра, которые на самом деле стоят за каждым `module_param`. Когда пишем

```
module_param(my_value, uint, 0644);
```

Ядро разворачивает это в структуру с указателями на функции `param_set_uint` и `param_get_uint`. Эти функции вызываются:

- `param_set` — когда ты пишешь в `/sys/module/.../parameters/...`
- `param_get` — когда ты читаешь из `/sys/module/.../parameters/...`

В ядре уже есть готовые функции для базовых типов:

Тип	set функция	get функция
-----	-------------	-------------

bool	param_set_bool	param_get_bool
int	param_set_int	param_get_int
uint	param_set_uint	param_get_uint
long	param_set_long	param_get_long
ulong	param_set_ulong	param_get_ulong
short	param_set_short	param_get_short
ushort	param_set_ushort	param_get_ushort
charp	param_set_charp	param_get_charp
invbool	param_set_invbool	param_get_invbool

Структура kernel_param Когда ты используешь module_param, ядро создает структуру:

```
struct kernel_param {
    const char *name;           // Имя параметра
    const struct kernel_param_ops *ops; // Указатель на функции
    u16 perm;                   // Права доступа (0644)
    s16 level;                  // Уровень для -EPROBE_DEFER
    union {
        void *arg;             // Указатель на переменную
        const struct kparam_string *str;
        const struct kparam_array *arr;
    };
};

struct kernel_param_ops {
    u16 flags;
    int (*set)(const char *val, const struct kernel_param *kp);
    int (*get)(char *buffer, const struct kernel_param *kp);
    void (*free)(void *arg);    // Для освобождения памяти (charp)
};
```

2.2. Как написать свои param_set и param_get

2.2.1. Параметр с валидацией диапазона

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/string.h>

MODULE_LICENSE("GPL");

static unsigned int my_value = 50;

// Наша функция установки (set)
```

```

static int param_set_my_value(const char *val, const struct kernel_param *kp)
{
    unsigned int new_val;
    int ret;

    // Используем стандартную функцию для парсинга числа
    ret = kstrtouint(val, 0, &new_val);
    if (ret < 0)
        return ret;

    // Валидация диапазона
    if (new_val < 10 || new_val > 100) {
        printk(KERN_WARNING "Value must be between 10 and 100, got %u\n", new_val);
        return -EINVAL;
    }

    // Если всё ок, записываем значение
    *(unsigned int *)kp->arg = new_val;
    printk(KERN_INFO "my_value set to %u\n", new_val);

    return 0;
}

// Наша функция чтения (get)
static int param_get_my_value(char *buffer, const struct kernel_param *kp)
{
    return sprintf(buffer, "%u\n", *(unsigned int *)kp->arg);
}

// Создаем свои операции
static const struct kernel_param_ops my_value_ops = {
    .set = param_set_my_value,
    .get = param_get_my_value,
};

// Используем module_param_cb вместо module_param
module_param_cb(my_value, &my_value_ops, &my_value, 0644);
MODULE_PARM_DESC(my_value, "Value between 10 and 100 (default: 50)");

static int __init my_init(void)
{
    printk(KERN_INFO "Module loaded, my_value = %u\n", my_value);
    return 0;
}

static void __exit my_exit(void)
{
    printk(KERN_INFO "Module unloaded\n");
}

module_init(my_init);

```

```
module_exit(my_exit);
```

Использование

```
sudo insmod my_module.ko my_value=75    # OK
sudo insmod my_module.ko my_value=200   # Ошибка!
cat /sys/module/my_module/parameters/my_value  # 75
echo 50 | sudo tee /sys/module/my_module/parameters/my_value  # OK
echo 200 | sudo tee /sys/module/my_module/parameters/my_value  # Ошибка!
```

2.2.2. Параметр с callback'ом (реакция на изменение)

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/random.h>

MODULE_LICENSE("GPL");

static unsigned int min_value = 0;
static unsigned int max_value = 100;
static u32 last_random = 0;

// Функция пересчета
static void recalculate(void)
{
    if (min_value > max_value) {
        printk(KERN_WARNING "Invalid range: min > max\n");
        return;
    }

    if (!rng_is_initialized())
        wait_for_random_bytes();

    last_random = get_random_u32_inclusive(min_value, max_value);
    printk(KERN_INFO "Recalculated: random = %u (range %u..%u)\n",
           last_random, min_value, max_value);
}

// Callback для min_value
static int set_min_value(const char *val, const struct kernel_param *kp)
{
    unsigned int new_val;
    int ret = kstrtouint(val, 0, &new_val);
    if (ret < 0)
        return ret;

    if (new_val > max_value) {
        printk(KERN_WARNING "min_value cannot be > max_value (%u)\n", max_value);
    }
}
```

```

        return -EINVAL;
    }

    *(unsigned int *)kp->arg = new_val;
    printk(KERN_INFO "min_value changed to %u, recalculating...\n", new_val);

    // ЗАПУСКАЕМ ПЕРЕЧЕТ!
    recalculate();

    return 0;
}

static int get_min_value(char *buffer, const struct kernel_param *kp)
{
    return sprintf(buffer, "%u\n", *(unsigned int *)kp->arg);
}

static const struct kernel_param_ops min_value_ops = {
    .set = set_min_value,
    .get = get_min_value,
};

// Callback для max_value
static int set_max_value(const char *val, const struct kernel_param *kp)
{
    unsigned int new_val;
    int ret = kstrtouint(val, 0, &new_val);
    if (ret < 0)
        return ret;

    if (new_val < min_value) {
        printk(KERN_WARNING "max_value cannot be < min_value (%u)\n", min_value);
        return -EINVAL;
    }

    *(unsigned int *)kp->arg = new_val;
    printk(KERN_INFO "max_value changed to %u, recalculating...\n", new_val);

    // ЗАПУСКАЕМ ПЕРЕЧЕТ!
    recalculate();

    return 0;
}

static int get_max_value(char *buffer, const struct kernel_param *kp)
{
    return sprintf(buffer, "%u\n", *(unsigned int *)kp->arg);
}

static const struct kernel_param_ops max_value_ops = {
    .set = set_max_value,

```

```

    .get = get_max_value,
};

// Регистрируем параметры с кастомными ops
module_param_cb(min_value, &min_value_ops, &min_value, 0644);
MODULE_PARM_DESC(min_value, "Minimum value (default: 0)");

module_param_cb(max_value, &max_value_ops, &max_value, 0644);
MODULE_PARM_DESC(max_value, "Maximum value (default: 100)");

// Обычный параметр для результата (только чтение через get)
static int get_result(char *buffer, const struct kernel_param *kp)
{
    return sprintf(buffer, "%u\n", last_random);
}

static const struct kernel_param_ops result_ops = {
    .set = param_set_uint, // Стандартный set (хотя мы не дадим писать)
    .get = get_result,
};

static u32 dummy_result = 0;
module_param_cb(result, &result_ops, &dummy_result, 0444); // Только чтение!
MODULE_PARM_DESC(result, "Last calculated random value (read-only)");

static int __init my_init(void)
{
    printk(KERN_INFO "Module loaded\n");
    recalculate();
    return 0;
}

static void __exit my_exit(void)
{
    printk(KERN_INFO "Module unloaded\n");
}

module_init(my_init);
module_exit(my_exit);

```

Использование

```

# Загружаем с начальными параметрами
sudo insmod my_module.ko min_value=10 max_value=50
# dmesg: Recalculated: random = 37 (range 10..50)

# Меняем параметр – пересчет запускается АВТОМАТИЧЕСКИ!
echo 20 | sudo tee /sys/module/my_module/parameters/min_value
# dmesg: min_value changed to 20, recalculating...
# dmesg: Recalculated: random = 42 (range 20..50)

```



```
# Читаем результат
cat /sys/module/my_module/parameters/result
# 42

# Пытаемся установить невалидное значение
echo 100 | sudo tee /sys/module/my_module/parameters/min_value
# dmesg: min_value cannot be > max_value (50)
# bash: echo: write error: Invalid argument
```

2.2.3. Когда использовать `param_set/param_get`

- Нужна валидация значений при установке
- Нужна реакция на изменение параметра (callback)
- Хочешь остаться в стандартном `/sys/module/.../parameters/`
- Не нужно создавать дополнительные файлы в `sysfs`

Не используй, когда:

- Нужны сложные структуры данных
- Нужны иерархические конфигурации
- Нужна двусторонняя связь с `user-space`

2.2.4. Ключевые моменты

- `module_param_cb(name, ops, arg, perm)` — макрос для регистрации параметра с кастомными функциями
- `kp → arg` — указатель на твою переменную, его нужно приводить к нужному типу
- `set` функция возвращает 0 при успехе или отрицательный код ошибки
- `get` функция должна вернуть количество записанных байт
- Можно комбинировать — часть параметров через `module_param`, часть через `module_param_cb`

2.2.5. Продвинутый трюк: массив с `callback`'ом

```
static int my_array[5];
static int array_count;

static int set_my_array(const char *val, const struct kernel_param *kp)
{
    // kp->arg указывает на struct kparam_array
    const struct kparam_array *arr = kp->arr;
    int *array = (int *)arr->elem;
    char *tmp, *token, *saveptr;
    int i = 0;
```

```

tmp = kstrdup(val, GFP_KERNEL);
if (!tmp) return -ENOMEM;

while ((token = strsep(&tmp, ",")) != NULL && i < arr->max) {
    if (kstrtoint(token, 0, &array[i]) < 0) {
        kfree(tmp);
        return -EINVAL;
    }
    i++;
}

*(int *)arr->num = i; // Обновляем счетчик
kfree(tmp);

printk(KERN_INFO "Array updated, count=%d\n", i);
recalculate();
return 0;
}

static int get_my_array(char *buffer, const struct kernel_param *kp)
{
    const struct kparam_array *arr = kp->arr;
    int *array = (int *)arr->elem;
    int count = *(int *)arr->num;
    int len = 0;

    for (int i = 0; i < count; i++) {
        len += sprintf(buffer + len, "%d", array[i]);
        if (i < count - 1)
            len += sprintf(buffer + len, ",");
    }
    len += sprintf(buffer + len, "\n");

    return len;
}

static const struct kernel_param_ops my_array_ops = {
    .set = set_my_array,
    .get = get_my_array,
};

// Используем module_param_array_cb
module_param_array_cb(my_array, int, &array_count, &my_array_ops, 0644);

```

3. Sysfs

3.1. Описание

Создает файлы в /sys/kernel с callback-функциями при чтении/записи.

```
#include <linux/module.h>
#include <linux/kobject.h>
#include <linux/sysfs.h>

MODULE_LICENSE("GPL");

static unsigned int my_value = 0;
static struct kobject *my_kobj;

// Вызывается при: cat /sys/kernel/my_module/value
static ssize_t value_show(struct kobject *kobj, struct kobj_attribute *attr, char
*buf)
{
    return sprintf(buf, "%u\n", my_value);
}

// Вызывается при: echo 42 | sudo tee /sys/kernel/my_module/value
static ssize_t value_store(struct kobject *kobj, struct kobj_attribute *attr,
                          const char *buf, size_t count)
{
    unsigned int new_val;
    if (kstrtouint(buf, 10, &new_val) < 0)
        return -EINVAL;

    my_value = new_val;
    printk(KERN_INFO "Value changed to %u\n", my_value);
    return count;
}

// Создаем атрибут (файл в sysfs)
static struct kobj_attribute value_attr = __ATTR(value, 0644, value_show,
value_store);

static struct attribute *attrs[] = { &value_attr.attr, NULL };
static struct attribute_group attr_group = { .attrs = attrs };

static int __init my_init(void)
{
    my_kobj = kobject_create_and_add("my_module", kernel_kobj);
    if (!my_kobj) return -ENOMEM;

    sysfs_create_group(my_kobj, &attr_group);
    printk(KERN_INFO "Module loaded\n");
    return 0;
}
```

```
static void __exit my_exit(void)
{
    sysfs_remove_group(my_kobj, &attr_group);
    kobject_put(my_kobj);
    printk(KERN_INFO "Module unloaded\n");
}

module_init(my_init);
module_exit(my_exit);
```

3.2. Использование

```
sudo insmod my_module.ko
cat /sys/kernel/my_module/value          # Чтение: 0
echo 42 | sudo tee /sys/kernel/my_module/value # Запись
cat /sys/kernel/my_module/value          # Чтение: 42
sudo rmmod my_module
```

4. /proc файлы

4.1. Описание

Создает файл в /proc/ для чтения/записи. Устаревший, но простой способ.

```
#include <linux/module.h>
#include <linux/proc_fs.h>
#include <linux/uaccess.h>

MODULE_LICENSE("GPL");

static unsigned int my_value = 0;
static struct proc_dir_entry *proc_file;

// Вызывается при: cat /proc/my_module
static ssize_t proc_read(struct file *file, char __user *buf, size_t count, loff_t
*ppos)
{
    char tmp[32];
    int len = sprintf(tmp, "%u\n", my_value);

    if (*ppos > 0 || count < len) return 0;
    if (copy_to_user(buf, tmp, len)) return -EFAULT;

    *ppos = len;
    return len;
}
```

```
// Вызывается при: echo 42 | sudo tee /proc/my_module
static ssize_t proc_write(struct file *file, const char __user *buf, size_t count,
loff_t *ppos)
{
    char tmp[32];
    unsigned int new_val;

    if (count >= sizeof(tmp)) return -EINVAL;
    if (copy_from_user(tmp, buf, count)) return -EFAULT;

    if (kstrtouint(tmp, 10, &new_val) < 0)
        return -EINVAL;

    my_value = new_val;
    printk(KERN_INFO "Value changed to %u\n", my_value);
    return count;
}

static const struct proc_ops proc_fops = {
    .proc_read = proc_read,
    .proc_write = proc_write,
};

static int __init my_init(void)
{
    proc_file = proc_create("my_module", 0644, NULL, &proc_fops);
    if (!proc_file) return -ENOMEM;

    printk(KERN_INFO "Module loaded\n");
    return 0;
}

static void __exit my_exit(void)
{
    proc_remove(proc_file);
    printk(KERN_INFO "Module unloaded\n");
}

module_init(my_init);
module_exit(my_exit);
```

4.2. Использование

```
sudo insmod my_module.ko
cat /proc/my_module          # Чтение: 0
echo 42 | sudo tee /proc/my_module  # Запись
cat /proc/my_module          # Чтение: 42
```

```
sudo rmmod my_module
```

4.3. /proc файлы для нескольких параметров

Есть два подхода к организации нескольких параметров через /proc.

4.3.1. Поддиректория с отдельными файлами (рекомендуемый)

Этот подход похож на sysfs — каждый параметр имеет свой файл в поддиректории /proc/my_module/.

```
#include <linux/module.h>
#include <linux/proc_fs.h>
#include <linux/seq_file.h>
#include <linux/uaccess.h>
#include <linux/string.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("user");
MODULE_DESCRIPTION("/proc multiple parameters example");

static unsigned int min_value = 0;
static unsigned int max_value = 100;
static int numbers[5] = {1, 2, 3, 4, 5};
static int numbers_count = 5;
static long long sum = 0;

static struct proc_dir_entry *proc_dir;

static void recalculate(void)
{
    sum = 0;
    for (int i = 0; i < numbers_count; i++)
        sum += numbers[i];
    printk(KERN_INFO "Recalculated: sum = %lld\n", sum);
}

// ===== min_value =====
static int min_value_show(struct seq_file *m, void *v)
{
    seq_printf(m, "%u\n", min_value);
    return 0;
}

static ssize_t min_value_write(struct file *file, const char __user *buf,
                               size_t count, loff_t *ppos)
{
    char tmp[32];
    unsigned int new_val;
```

```

    if (count >= sizeof(tmp))
        return -EINVAL;
    if (copy_from_user(tmp, buf, count))
        return -EFAULT;

    tmp[count] = '\0';
    if (kstrtouint(tmp, 10, &new_val) < 0)
        return -EINVAL;

    if (new_val > max_value) {
        printk(KERN_WARNING "min_value cannot be > max_value\n");
        return -EINVAL;
    }

    min_value = new_val;
    printk(KERN_INFO "min_value changed to %u\n", min_value);
    recalculate();
    return count;
}

static int min_value_open(struct inode *inode, struct file *file)
{
    return single_open(file, min_value_show, NULL);
}

static const struct proc_ops min_value_fops = {
    .proc_open    = min_value_open,
    .proc_read    = seq_read,
    .proc_write   = min_value_write,
    .proc_lseek   = seq_lseek,
    .proc_release = single_release,
};

// ===== max_value =====
static int max_value_show(struct seq_file *m, void *v)
{
    seq_printf(m, "%u\n", max_value);
    return 0;
}

static ssize_t max_value_write(struct file *file, const char __user *buf,
                               size_t count, loff_t *ppos)
{
    char tmp[32];
    unsigned int new_val;

    if (count >= sizeof(tmp))
        return -EINVAL;
    if (copy_from_user(tmp, buf, count))
        return -EFAULT;

```

```

    tmp[count] = '\0';
    if (kstrtouint(tmp, 10, &new_val) < 0)
        return -EINVAL;

    if (new_val < min_value) {
        printk(KERN_WARNING "max_value cannot be < min_value\n");
        return -EINVAL;
    }

    max_value = new_val;
    printk(KERN_INFO "max_value changed to %u\n", max_value);
    recalculate();
    return count;
}

static int max_value_open(struct inode *inode, struct file *file)
{
    return single_open(file, max_value_show, NULL);
}

static const struct proc_ops max_value_fops = {
    .proc_open    = max_value_open,
    .proc_read    = seq_read,
    .proc_write   = max_value_write,
    .proc_lseek   = seq_lseek,
    .proc_release = single_release,
};

// ===== numbers (массив) =====
static int numbers_show(struct seq_file *m, void *v)
{
    seq_printf(m, "count=%d values=", numbers_count);
    for (int i = 0; i < numbers_count; i++) {
        seq_printf(m, "%d", numbers[i]);
        if (i < numbers_count - 1)
            seq_printf(m, ",");
    }
    seq_printf(m, "\n");
    return 0;
}

static ssize_t numbers_write(struct file *file, const char __user *buf,
                             size_t count, loff_t *ppos)
{
    char tmp[128];
    char *token;
    char *str_ptr;
    int i = 0;

    if (count >= sizeof(tmp))

```



```

        return -EINVAL;
    if (copy_from_user(tmp, buf, count))
        return -EFAULT;

    tmp[count] = '\0';

    // Убираем перевод строки
    char *newline = strchr(tmp, '\n');
    if (newline)
        *newline = '\0';

    // Парсим строку вида: "10,20,30,40,50"
    str_ptr = tmp;
    numbers_count = 0;

    while ((token = strsep(&str_ptr, ",")) != NULL && i < 5) {
        if (strlen(token) == 0)
            continue;
        if (kstrtoint(token, 10, &numbers[i]) < 0)
            return -EINVAL;
        i++;
    }
    numbers_count = i;

    printk(KERN_INFO "numbers updated, count=%d\n", numbers_count);
    recalculate();
    return count;
}

static int numbers_open(struct inode *inode, struct file *file)
{
    return single_open(file, numbers_show, NULL);
}

static const struct proc_ops numbers_fops = {
    .proc_open    = numbers_open,
    .proc_read    = seq_read,
    .proc_write   = numbers_write,
    .proc_lseek   = seq_lseek,
    .proc_release = single_release,
};

// ===== result (только для чтения) =====
static int result_show(struct seq_file *m, void *v)
{
    seq_printf(m, "min=%u max=%u sum=%lld count=%d\n",
               min_value, max_value, sum, numbers_count);
    return 0;
}

static int result_open(struct inode *inode, struct file *file)

```

```

{
    return single_open(file, result_show, NULL);
}

static const struct proc_ops result_fops = {
    .proc_open    = result_open,
    .proc_read    = seq_read,
    .proc_lseek   = seq_lseek,
    .proc_release = single_release,
};

// ===== Инициализация =====
static int __init my_init(void)
{
    proc_dir = proc_mkdir("my_module", NULL);
    if (!proc_dir) {
        printk(KERN_ERR "Failed to create /proc/my_module\n");
        return -ENOMEM;
    }

    proc_create("min_value", 0644, proc_dir, &min_value_fops);
    proc_create("max_value", 0644, proc_dir, &max_value_fops);
    proc_create("numbers",   0644, proc_dir, &numbers_fops);
    proc_create("result",    0444, proc_dir, &result_fops);

    printk(KERN_INFO "Module loaded. Use /proc/my_module/\n");
    recalculate();
    return 0;
}

static void __exit my_exit(void)
{
    remove_proc_entry("min_value", proc_dir);
    remove_proc_entry("max_value", proc_dir);
    remove_proc_entry("numbers",   proc_dir);
    remove_proc_entry("result",    proc_dir);
    remove_proc_entry("my_module", NULL);

    printk(KERN_INFO "Module unloaded\n");
}

module_init(my_init);
module_exit(my_exit);

```

4.3.2. Использование

```

sudo insmod my_module.ko

# Смотрим структуру

```

```
ls /proc/my_module/
# min_value max_value numbers result

# Читаем параметры
cat /proc/my_module/min_value # 0
cat /proc/my_module/max_value # 100
cat /proc/my_module/numbers   # count=5 values=1,2,3,4,5
cat /proc/my_module/result     # min=0 max=100 sum=15 count=5

# Меняем параметры
echo 10 | sudo tee /proc/my_module/min_value
echo 200 | sudo tee /proc/my_module/max_value
echo "10,20,30" | sudo tee /proc/my_module/numbers

# Проверяем результат
cat /proc/my_module/result      # min=10 max=200 sum=60 count=3

sudo rmmod my_module
```

4.4. Один файл с командами (упрощенный)

Все параметры управляются через один файл `/proc/my_module`. Команды передаются в формате `ключ=значение`.

```
#include <linux/module.h>
#include <linux/proc_fs.h>
#include <linux/seq_file.h>
#include <linux/uaccess.h>
#include <linux/string.h>

MODULE_LICENSE("GPL");

static unsigned int min_value = 0;
static unsigned int max_value = 100;
static int numbers[5] = {1, 2, 3, 4, 5};
static int numbers_count = 5;
static long long sum = 0;
static struct proc_dir_entry *proc_file;

static void recalculate(void)
{
    sum = 0;
    for (int i = 0; i < numbers_count; i++)
        sum += numbers[i];
}

// Чтение: выводим ВСЕ параметры
static int my_proc_show(struct seq_file *m, void *v)
{
```

```

seq_printf(m, "min_value=%u\n", min_value);
seq_printf(m, "max_value=%u\n", max_value);
seq_printf(m, "numbers=");
for (int i = 0; i < numbers_count; i++) {
    seq_printf(m, "%d", numbers[i]);
    if (i < numbers_count - 1) seq_printf(m, ",");
}
seq_printf(m, "\n");
seq_printf(m, "sum=%lld\n", sum);
return 0;
}

// Запись: парсим команды вида "ключ=значение"
static ssize_t my_proc_write(struct file *file, const char __user *buf,
                             size_t count, loff_t *ppos)
{
    char tmp[256];
    char *eq_sign;

    if (count >= sizeof(tmp))
        return -EINVAL;
    if (copy_from_user(tmp, buf, count))
        return -EFAULT;

    tmp[count] = '\0';
    // Убираем перевод строки
    char *newline = strchr(tmp, '\n');
    if (newline) *newline = '\0';

    // Ищем знак '='
    eq_sign = strchr(tmp, '=');
    if (!eq_sign) {
        printk(KERN_WARNING "Format: key=value\n");
        return -EINVAL;
    }

    *eq_sign = '\0'; // Разделяем ключ и значение
    char *key = tmp;
    char *value = eq_sign + 1;

    // Обрабатываем команды
    if (strcmp(key, "min_value") == 0) {
        unsigned int val;
        if (kstrtouint(value, 10, &val) < 0)
            return -EINVAL;
        if (val > max_value) return -EINVAL;
        min_value = val;
        printk(KERN_INFO "min_value = %u\n", min_value);
    } else if (strcmp(key, "max_value") == 0) {
        unsigned int val;

```

```

        if (kstrtoint(value, 10, &val) < 0)
            return -EINVAL;
        if (val < min_value) return -EINVAL;
        max_value = val;
        printk(KERN_INFO "max_value = %u\n", max_value);

    } else if (strcmp(key, "numbers") == 0) {
        char *token, *saveptr;
        int i = 0;
        char *value_copy = kstrdup(value, GFP_KERNEL);
        if (!value_copy) return -ENOMEM;

        numbers_count = 0;
        while ((token = strsep(&value_copy, ",")) != NULL && i < 5) {
            if (kstrtoint(token, 10, &numbers[i]) < 0) {
                kfree(value_copy);
                return -EINVAL;
            }
            i++;
        }
        numbers_count = i;
        kfree(value_copy);
        printk(KERN_INFO "numbers updated, count=%d\n", numbers_count);

    } else {
        printk(KERN_WARNING "Unknown key: %s\n", key);
        return -EINVAL;
    }

    recalculate();
    return count;
}

static int my_proc_open(struct inode *inode, struct file *file)
{
    return single_open(file, my_proc_show, NULL);
}

static const struct proc_ops my_proc_fops = {
    .proc_open    = my_proc_open,
    .proc_read    = seq_read,
    .proc_write   = my_proc_write,
    .proc_lseek   = seq_lseek,
    .proc_release = single_release,
};

static int __init my_init(void)
{
    proc_file = proc_create("my_module", 0644, NULL, &my_proc_fops);
    if (!proc_file) return -ENOMEM;
}

```

```

    printk(KERN_INFO "Module loaded. Use /proc/my_module\n");
    recalculate();
    return 0;
}

static void __exit my_exit(void)
{
    proc_remove(proc_file);
    printk(KERN_INFO "Module unloaded\n");
}

module_init(my_init);
module_exit(my_exit);

```

4.4.1. Использование

```

sudo insmod my_module.ko

# Читаем BCE параметры сразу
cat /proc/my_module
# min_value=0
# max_value=100
# numbers=1,2,3,4,5
# sum=15

# Меняем параметры по одному
echo "min_value=10" | sudo tee /proc/my_module
echo "max_value=200" | sudo tee /proc/my_module
echo "numbers=10,20,30" | sudo tee /proc/my_module

# Проверяем
cat /proc/my_module
# min_value=10
# max_value=200
# numbers=10,20,30
# sum=60

sudo rmmod my_module

```

5. debugfs (для отладки)

5.1. Описание

Простейший способ для отладочных целей. Автоматическое создание файлов.

```
#include <linux/module.h>
```

```
#include <linux/debugfs.h>

MODULE_LICENSE("GPL");

static unsigned int my_value = 0;
static u32 my_counter = 0;
static struct dentry *debug_dir;

static int __init my_init(void)
{
    // Создаем директорию в /sys/kernel/debug/
    debug_dir = debugfs_create_dir("my_module", NULL);
    if (!debug_dir) return -ENOMEM;

    // Автоматически создаем файлы для переменных
    debugfs_create_u32("value", 0644, debug_dir, &my_value);
    debugfs_create_u32("counter", 0644, debug_dir, &my_counter);

    printk(KERN_INFO "Module loaded\n");
    return 0;
}

static void __exit my_exit(void)
{
    debugfs_remove_recursive(debug_dir);
    printk(KERN_INFO "Module unloaded\n");
}

module_init(my_init);
module_exit(my_exit);
```

5.2. Использование

```
sudo insmod my_module.ko
cat /sys/kernel/debug/my_module/value    # Чтение: 0
echo 42 | sudo tee /sys/kernel/debug/my_module/value # Запись
cat /sys/kernel/debug/my_module/counter  # Чтение: 0
sudo rmmod my_module
```

6. Character Device + ioctl

6.1. Описание

Создает устройство /dev/my_device для обмена через системные вызовы ioctl().

```
#include <linux/module.h>
```

```

#include <linux/fs.h>
#include <linux/cdev.h>
#include <linux/uaccess.h>

MODULE_LICENSE("GPL");

#define IOCTL_SET_VALUE _IOW('M', 1, unsigned int)
#define IOCTL_GET_VALUE _IOR('M', 2, unsigned int)

static unsigned int my_value = 0;
static dev_t dev_num;
static struct cdev my_cdev;
static struct class *my_class;

// Обработка ioctl запросов
static long device_ioctl(struct file *file, unsigned int cmd, unsigned long arg)
{
    switch(cmd) {
        case IOCTL_SET_VALUE:
            if (copy_from_user(&my_value, (void __user *)arg, sizeof(my_value)))
                return -EFAULT;
            printk(KERN_INFO "Value set to %u\n", my_value);
            return 0;

        case IOCTL_GET_VALUE:
            if (copy_to_user((void __user *)arg, &my_value, sizeof(my_value)))
                return -EFAULT;
            return 0;
    }
    return -EINVAL;
}

static const struct file_operations fops = {
    .owner = THIS_MODULE,
    .unlocked_ioctl = device_ioctl,
};

static int __init my_init(void)
{
    // Регистрируем символьное устройство
    if (alloc_chrdev_region(&dev_num, 0, 1, "my_device"))
        return -ENOMEM;

    my_class = class_create("my_class");
    if (IS_ERR(my_class)) {
        unregister_chrdev_region(dev_num, 1);
        return PTR_ERR(my_class);
    }

    device_create(my_class, NULL, dev_num, NULL, "my_device");
}

```



```

cdev_init(&my_cdev, &fops);
cdev_add(&my_cdev, dev_num, 1);

printk(KERN_INFO "Module loaded, use /dev/my_device\n");
return 0;
}

static void __exit my_exit(void)
{
    cdev_del(&my_cdev);
    device_destroy(my_class, dev_num);
    class_destroy(my_class);
    unregister_chrdev_region(dev_num, 1);
    printk(KERN_INFO "Module unloaded\n");
}

module_init(my_init);
module_exit(my_exit);

```

6.2. Пользовательская программа

```

#include <stdio.h>
#include <fcntl.h>
#include <sys/ioctl.h>

#define IOCTL_SET_VALUE _IOW('M', 1, unsigned int)
#define IOCTL_GET_VALUE _IOR('M', 2, unsigned int)

int main()
{
    int fd = open("/dev/my_device", O_RDWR);
    if (fd < 0) {
        perror("open");
        return 1;
    }

    unsigned int value = 42;
    ioctl(fd, IOCTL_SET_VALUE, &value);

    unsigned int read_value;
    ioctl(fd, IOCTL_GET_VALUE, &read_value);
    printf("Read value: %u\n", read_value);

    close(fd);
    return 0;
}

```

6.3. Использование

```
sudo insmod my_module.ko
gcc user_program.c -o user_program
sudo ./user_program          # Вывод: Read value: 42
dmesg | tail                  # Value set to 42
sudo rmmod my_module
```

7. Netlink сокет

7.1. Описание

```
#include <linux/module.h>
#include <net/netlink.h>
#include <net/sock.h>

MODULE_LICENSE("GPL");

#define MY_NETLINK_FAMILY 31 // Произвольный номер протокола

static unsigned int my_value = 0;
static struct sock *nl_sk;

// Обработка входящих сообщений
static void netlink_recv(struct sk_buff *skb)
{
    struct nlmsghdr *nlh;
    unsigned int *new_val;

    nlh = nlmsg_hdr(skb);
    new_val = (unsigned int *)nlmsg_data(nlh);

    my_value = *new_val;
    printk(KERN_INFO "Received value: %u\n", my_value);

    // Отправляем ответ
    nlh = nlmsg_new(NLMSG_ALIGN(sizeof(unsigned int)), GFP_KERNEL);
    if (nlh) {
        unsigned int *reply = nlmsg_data(nlmsg_put(nlh, 0, 0, 0, sizeof(unsigned int),
0));
        *reply = my_value;
        NETLINK_CB(nl_sk->sk_socket->sk).portid = nlh->nlmsg_pid;
        netlink_unicast(nl_sk, skb, nlh->nlmsg_pid, MSG_DONTWAIT);
    }
}

static int __init my_init(void)
```

```

{
    // Создаем netlink сокет
    nl_sk = netlink_kernel_create(&init_net, MY_NETLINK_FAMILY, NULL);
    if (!nl_sk) return -ENOMEM;

    printk(KERN_INFO "Netlink module loaded\n");
    return 0;
}

static void __exit my_exit(void)
{
    netlink_kernel_release(nl_sk);
    printk(KERN_INFO "Netlink module unloaded\n");
}

module_init(my_init);
module_exit(my_exit);

```

7.2. Пользовательская программа

```

#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <linux/netlink.h>

#define MY_NETLINK_FAMILY 31

int main()
{
    int sock;
    struct sockaddr_nl src_addr, dest_addr;
    struct nlmsghdr *nlh;
    struct msghdr msg;
    struct iovec iov;
    unsigned int value = 42;

    sock = socket(AF_NETLINK, SOCK_RAW, MY_NETLINK_FAMILY);

    memset(&src_addr, 0, sizeof(src_addr));
    src_addr.nl_family = AF_NETLINK;
    src_addr.nl_pid = getpid();
    bind(sock, (struct sockaddr*)&src_addr, sizeof(src_addr));

    memset(&dest_addr, 0, sizeof(dest_addr));
    dest_addr.nl_family = AF_NETLINK;
    dest_addr.nl_pid = 0; // Ядро
    dest_addr.nl_groups = 0;

```

```

nlh = (struct nlmsghdr *)malloc(NLMSG_SPACE(sizeof(unsigned int)));
memset(nlh, 0, NLMSG_SPACE(sizeof(unsigned int)));
nlh->nlmsg_len = NLMSG_SPACE(sizeof(unsigned int));
nlh->nlmsg_pid = getpid();
nlh->nlmsg_flags = 0;
*(unsigned int *)NLMSG_DATA(nlh) = value;

iov.iov_base = (void *)nlh;
iov.iov_len = nlh->nlmsg_len;

memset(&msg, 0, sizeof(msg));
msg.msg_name = (void *)&dest_addr;
msg.msg_namelen = sizeof(dest_addr);
msg.msg_iov = &iov;
msg.msg_iovlen = 1;

sendmsg(sock, &msg, 0);
printf("Sent value: %u\n", value);

close(sock);
free(nlh);
return 0;
}

```

7.3. Использование

```

sudo insmod my_module.ko
gcc user_program.c -o user_program
sudo ./user_program          # Sent value: 42
dmesg | tail                 # Received value: 42
sudo rmmod my_module

```

8. configfs (для сложной конфигурации)

8.1. Описание

Позволяет создавать/удалять объекты через файловую систему. Используется для сложных конфигураций.

```

#include <linux/module.h>
#include <linux/configfs.h>

MODULE_LICENSE("GPL");

// Структура для хранения конфигурации
struct my_config {

```

```

    struct config_group group;
    unsigned int value;
};

// Макрос для получения my_config из config_group
#define to_my_config(group) container_of(group, struct my_config, group)

// Атрибуты конфигурации
struct my_attr {
    struct configfs_attribute attr;
    ssize_t (*show)(struct my_config *, char *);
    ssize_t (*store)(struct my_config *, const char *, size_t);
};

static ssize_t my_value_show(struct config_item *item, char *page)
{
    struct my_config *cfg = container_of(to_config_group(item), struct my_config,
group);
    return sprintf(page, "%u\n", cfg->value);
}

static ssize_t my_value_store(struct config_item *item, const char *page, size_t
count)
{
    struct my_config *cfg = container_of(to_config_group(item), struct my_config,
group);
    unsigned int val;

    if (kstrtouint(page, 10, &val) < 0)
        return -EINVAL;

    cfg->value = val;
    printk(KERN_INFO "Config value set to %u\n", val);
    return count;
}

static struct my_attr my_value_attr = {
    .attr = { .ca_name = "value", .ca_mode = 0644, .ca_owner = THIS_MODULE },
    .show = my_value_show,
    .store = my_value_store,
};

static struct configfs_attribute *my_attrs[] = {
    &my_value_attr.attr,
    NULL,
};

// Операции для подгрупп
static struct configfs_group_operations my_group_ops = {
    // Можно добавить создание/удаление подгрупп
};

```

```

static struct config_item_type my_item_type = {
    .ct_group_ops = &my_group_ops,
    .ct_attrs = my_attrs,
    .ct_owner = THIS_MODULE,
};

static struct configfs_subsystem my_subsys = {
    .su_group = {
        .cg_item = {
            .ci_namebuf = "my_module",
            .ci_type = &my_item_type,
        },
    },
};

static int __init my_init(void)
{
    config_group_init(&my_subsys.su_group);
    mutex_init(&my_subsys.su_mutex);

    if (configfs_register_subsystem(&my_subsys)) {
        printk(KERN_ERR "Failed to register configfs\n");
        return -ENOMEM;
    }

    printk(KERN_INFO "Configfs module loaded\n");
    return 0;
}

static void __exit my_exit(void)
{
    configfs_unregister_subsystem(&my_subsys);
    printk(KERN_INFO "Configfs module unloaded\n");
}

module_init(my_init);
module_exit(my_exit);

```

8.2. Использование

```

sudo insmod my_module.ko
ls /sys/kernel/config/my_module/      # Просмотр структуры
cat /sys/kernel/config/my_module/value # Чтение: 0
echo 42 | sudo tee /sys/kernel/config/my_module/value # Запись
cat /sys/kernel/config/my_module/value # Чтение: 42
sudo rmmod my_module

```